# Übungsblatt 9: Multicore & Finale

**Mikrocomputertechnik 3 · Zustände, Atomics und Systemrechnungen auf Papier**

## Ziel und Arbeitsweise

Sechs Pflichtaufgaben, vollständig auf Papier lösbar. Geben Sie Modellannahmen, Zwischenwerte und Einheiten an. Es sind keine Hardwaremessungen oder neue Prüfungsformalien enthalten.

**Bearbeitungszeit:** etwa 75–90 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht: Aufgaben 1–2 (10 min), Aufgabe 3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Die Vorlesungsphasen enthalten Lösungsbesprechungen.

MESI bedeutet Modified, Exclusive, Shared, Invalid. Hart ist eine Hardware-Ausführungsinstanz, Thread ein Softwareablauf. RMW heißt Read-Modify-Write, Lesen, Ändern und Schreiben. LR/SC steht für Load-Reserved/Store-Conditional. KiB = 1024 Byte, MHz = $10^6$ Takte je Sekunde.

## Aufgabe 1: MESI mit Datenwerten (6 Punkte)

Lehrmodell K: zwei Datencaches, Write-Back, Write-Allocate, stabile MESI-Zustände. Jede Zeile umfasst 64 Byte. $L$ ist die untere kohärente Speicherebene; anfangs $x = 0$, beide Caches I. Beim Fremdread auf M werden die aktuellen Daten im Modell nach $L$ übertragen und beide Kopien S. Geänderte Daten werden vor M-Auswurf zurückgeschrieben.

1. Ergänzen Sie Zustände mit Werten und den unteren Stand nach jedem abgeschlossenen Schritt. (3 P)
2. Warum darf $L$ nach dem ersten Write noch null sein? Darf Kern 1 beim folgenden Read trotzdem null liefern? (1 P)
3. Welche Aktionen müssen beim Write auf S vor dem stabilen Abschluss erfolgen? (1 P)
4. Separater Fall: Beide Caches halten S:5, dann wirft Kern 0 seine saubere Kopie aus. Muss Kern 1 automatisch E werden? Begründen Sie. (1 P)

| Aktion | Kern 0 | Kern 1 | $L$ |
|---|---|---|---|
| Start | I | I | 0 |
| Kern 0 liest $x$ | | | |
| Kern 1 liest $x$ | | | |
| Kern 0 schreibt $x = 5$ | | | |
| Kern 1 liest $x$ | | | |
| Kern 1 schreibt $x = 7$ | | | |
| Kern 1 wirft seine Zeile aus | | | |
| Kern 0 liest $x$ | | | |

**Ihre Begründungen:**

## Aufgabe 2: False Sharing und Besitzerwechsel (4 Punkte)

Cachezeile 64 Byte, Objektgröße je 4 Byte, ausgerichtete Basis `0x20001000`. Kern 0 schreibt nur Objekt $a$ bei Offset 0, Kern 1 nur Objekt $b$ bei Offset 4. Die Objekte sind softwareseitig getrennt zugeteilt.

1. Geben Sie beide Adressen und den vollständigen gemeinsamen Zeilenbereich an. (1 P)
2. Anfangs besitzt Kern 0 die Zeile bereits M. Schreibfolge: Kern 0, Kern 0, Kern 1, Kern 1, Kern 0. Wie oft wechselt die exklusive Zuständigkeit zwischen Kernen? Warum nicht bei jedem Write? (1 P)
3. $b$ wird nun bei Offset 64 abgelegt. Bestimmen Sie die neue Adresse und erklären Sie, weshalb bloße Basisausrichtung bei Offset 4 nicht genügt. (1 P)
4. Würde solcher Abstand ein verlorenes Update zweier Schreiber auf denselben ungeschützten Zähler reparieren? (1 P)

**Ihre Zeilenskizze und Besitzerfolge:**

## Aufgabe 3: Lost Update und LR/SC (6 Punkte)

1. Im Hardwaremodell lesen beide Harts Zähler null, erhöhen ihren lokalen Registerwert und schreiben zurück. Notieren Sie eine vollständige Schrittfolge, den Endwert und warum MESI ihn nicht korrigiert. (2 P)
2. Was bedeutet derselbe Konflikt bei einem gewöhnlichen ungeschützten C-`int`? Darf man die Endwerte eins oder zwei als garantierte C11-Liste angeben? Würden getrennte atomare Loads und Stores die Gruppenwirkung schon liefern? (1 P)
3. Binäres Schloss: frei null, belegt eins. Beide LR lesen null. SC von Hart 0 versucht eins und gewinnt. Danach versucht SC von Hart 1 ebenfalls eins und scheitert. Füllen Sie den Ablauf einschließlich Rückgabestatus aus. (2 P)
4. Darf Hart 1 nach einem neuen LR mit Wert eins den kritischen Abschnitt betreten? Muss jeder SC-Fehler einen fremden Store beweisen? (1 P)

| Ereignis | gelesener Wert / Status | Schloss danach |
|---|---|---|
| LR Hart 0 | | |
| LR Hart 1 | | |
| SC Hart 0 mit eins, Erfolg | | |
| SC Hart 1 mit eins, Fehler | | |
| Neuer LR Hart 1 | | |
| Hart 0 gibt frei | | |

**Ihre Registerfolge und Interpretation:**

## Aufgabe 4: Ordnung und Fortschritt (6 Punkte)

1. Ein Produzent schreibt einmalig gewöhnliche Nutzdaten `data = 42` und danach atomar `ready = 1` mit Release. Ein Konsument liest genau diese Eins mit Acquire und danach `data`. Nennen Sie Ergebnis und nötige Voraussetzungen. (2 P)
2. Separates wohldefiniertes Gegenmodell: Sowohl `data` als auch `ready` sind atomare Objekte, alle Zugriffe relaxed. Darf der Konsument bei ready eins noch data null beobachten? Warum wäre gewöhnliche ungeschützte Payload ein anderer Fall? (1 P)
3. Kern 0 hält Lock A und wartet auf B, Kern 1 hält B und wartet auf A. Skizzieren Sie den Wartekreis und eine gemeinsame Reihenfolge, die genau diesen Kreis verhindert. Dürfte ein Timeout ein fremdes Lock löschen? (2 P)
4. Auf demselben Hart hält ein Kontext L und wird unterbrochen. Die ISR spinnt auf L. Warum hilft eine weitere atomare Übernahmeoperation nicht? Hält eine lokale IRQ-Sperre andere Harts fern? (1 P)

ISR bedeutet Interrupt Service Routine, IRQ Interrupt Request. Acquire und Release sind Übernahme- und Freigabeordnung.

**Ihre Übergabebeziehung und Wartegraphen:**

## Aufgabe 5: Speedup, Effizienz und Verteilung (4 Punkte)

Feste Aufgabe: Ein-Kern-Zeit 100 Zeiteinheiten, serieller Anteil $s = 0{,}2$, paralleler Anteil 0,8. Zunächst perfekte Verteilung auf gleiche Kerne, kein Zusatzaufwand. $S(N) = 1/(s + (1-s)/N)$, $E(N) = S(N)/N$.

1. Berechnen Sie Zeit, Speedup und Effizienz bei vier Kernen. (1 P)
2. Berechnen Sie Speedup und Effizienz bei hundert Kernen sowie den Grenzfaktor. Wieso kann der Speedup nahe am Grenzwert und die Effizienz klein sein? (1 P)
3. Bei vier Kernen fallen nun fünf zusätzliche Zeiteinheiten an. Berechnen Sie neue Zeit und Speedup. Welchen normierten Wert hat $h(4)$ in der erweiterten Formel? (1 P)
4. Separat ohne diesen Zusatzaufwand: Die achtzig parallelen Einheiten sind als $10, 10, 10, 50$ verteilt. Bestimmen Sie parallele Abschlusszeit, Gesamtzeit und Speedup. (1 P)

**Ihre Zeitbalken und Rechnungen:**

## Aufgabe 6: Systemrechnungen zum Abschluss (6 Punkte)

1. Direkt abgebildeter Cache: 4 KiB Nutzdaten, 64 Byte je Zeile, 32-Bit-Byteadresse. Bestimmen Sie Satzanzahl, Offset-, Index- und Tagbits. Zerlegen Sie `0x20001084` und geben Sie die Zeilenbasis an. (2 P)
2. Hitzeit zwei Takte, Hitrate 0,9, zusätzliche Misspenalty vierzig Takte. Bestimmen Sie AMAT und die Zeit bei 100 MHz. (1 P)
3. Lehrtimer: Eingang 1 MHz, Teiler zehn, Zähler null bis neunhundertneunundneunzig. High bei null bis zweihundertneunundvierzig. Bestimmen Sie PWM-Frequenz, Periodendauer, aktive Takte und Duty Cycle. Warum darf ein Endregisterwert nicht ungeprüft als Periodenlänge verwendet werden? (2 P)
4. Ein DMA-Gerät schreibt ein Sensorfenster in einen Puffer. CPU-Caches sind untereinander kohärent; der Gerätevertrag ist noch offen. Darf die CPU daraus automatisch DMA-Kohärenz und ein fertiges Fenster folgern? Nennen Sie die gesondert nötigen Prüfungen. (1 P)

AMAT heißt Average Memory Access Time, PWM Pulse Width Modulation, DMA Direct Memory Access, CPU Central Processing Unit.

**Ihre Adresszerlegung, Zeitrechnung und Übergabebedingungen:**

### Optional: korrekte Hostbeispiele ausführen

Die Datei `src/lab09_atomics.c` nutzt C11-Atomics und POSIX Threads. Vom Repository-Hauptordner aus:

```
gcc -std=c11 -O2 -Wall -Wextra -Werror -pedantic -pthread \
    labs/src/lab09_atomics.c -o /tmp/lab09_atomics
/tmp/lab09_atomics
```

Erwartete Ausgabe:

```
Teilsummen: 10 26, gesamt 36
Getrennte atomare Zugriffe: Endwert 1
Atomare RMW: 40000
Geschuetzter Zaehler: 40000
Veroeffentlichte Daten: 42
Alle Prueffaelle bestanden.
```

1. Finden Sie die getrennt zugeteilten Ergebnisplätze und den Abschluss durch `pthread_join`.
2. Warum ist die serielle Lost-Update-Nachbildung wohldefiniert, ein ausgeführtes gewöhnliches racy C-`int`-Experiment aber kein Beweis?
3. Begründen Sie, warum relaxed für den reinen Zähler genügt, die Datenveröffentlichung dagegen Release/Acquire verwendet.
4. Optional: mit `-O1 -g -fsanitize=address,undefined` prüfen. Ein solcher Lauf beweist weder Fairness noch MESI-Geschwindigkeit.